

驱动到底要做什么

—— 驱动的三件事,以及它和"应用"的分工

MPU-6050 IIO 驱动学习 · 概念梳理

2026 年 9 月

一、一句话:驱动是"硬件"和"用户"之间的桥

很多初学者以为"驱动 = 把硬件信息注册一下,让内核能找到硬件"。这是最大的误区。"描述硬件在哪"是设备树的事;驱动真正的职责,是把一颗**躺在总线上、谁也用不了的芯片**,变成一个**用户能用的服务**。

驱动的本质:它是"硬件"与"用户"之间的**桥**。一个完全不懂 I2C、不懂芯片寄存器的程序,只要读一个文件就能拿到传感器数据——中间那座桥,就是驱动。

二、先分清三个角色:设备 / 驱动 / 应用

角色	是什么	谁负责 / 在哪
设备 device	描述"哪里有个什么硬件"(在哪条总线、地址、名字)	设备树(真板子)/ new_device(模拟);不是驱动
驱动 driver	操作这个硬件,并开一个接口让用户能用它	你写的代码,运行在 内核态
应用 application	使用这个硬件(读数据、做业务)	用户程序(cat / python / App),运行在 用户态

"让内核找到硬件"只是**设备 + 匹配机制**干的事,它只是让驱动的 `probe` 被调用——这是**开始**,不是结束。

三、驱动要做的三件事

把一颗芯片变成可用的服务,驱动要依次做三件事(图 1)。



图 1 驱动要做的三件事

① 匹配:让内核把硬件“交给你”

声明“我认得哪些设备”(`id_table` 按名字 / `of_match_table` 按设备树 `compatible`)。总线上出现匹配的设备 → 内核自动调用你的 `probe()` 。注意:描述“硬件在哪”是设备树/`new_device` 干的,不是驱动。

② 初始化:把硬件弄到能工作的状态

建立读写通道(`regmap`)、读 `WHO_AM_I` 确认芯片身份、唤醒芯片(清 `sleep` 位)、配置量程/采样率等。这步做完,芯片“活了”,但用户还用不了它。

③ 开接口:让用户能用这个硬件(驱动的目的!)

注册到子系统(如 `IIO`)、定义通道(能测什么量)、实现回调 `read_raw` (用户来读时去芯片取数据返回)。注册后会自动生成 `/sys/bus/iio/devices/iio:device0/in_accel_x_raw` 等文件,用户 `cat` 就能拿数据。

只做 ① 等于"检测到了芯片但没人能用它";只有做到 ③,这颗传感器才真正"可用"。这正是驱动存在的意义。

四、驱动 vs 应用:它们是分开的!

常见疑问:"是不是把驱动和应用写在一起了?"

答:没有。我们从头到尾只写了驱动,一行应用都没写。

驱动和应用是两个分开的程序,运行在两个不同的世界(图 2):

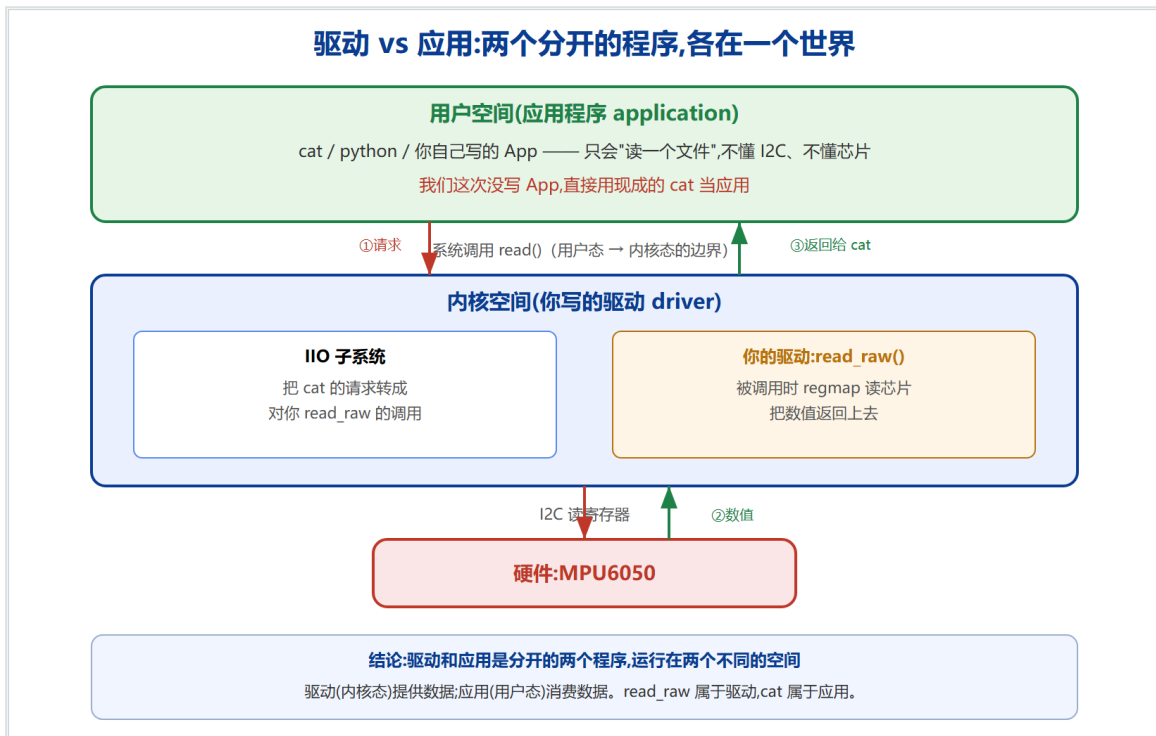


图 2 驱动(内核态)与应用(用户态)的分工与数据流

- **驱动(内核态):**我们写的 `mpu6050_iio.c`。它只负责"把数据准备好,放到 `sysfs` 文件里"。`read_raw()` 是驱动这一侧的回调(响应"有人来读了")。
- **应用(用户态):**使用数据的程序。我们用的 `cat` 就是应用——Linux 自带的通用工具。我们没写应用,直接借 `cat` 来读。

4.1 和你熟悉的 LED 例子对照

在经典字符设备里,你通常会写两个文件:

- `led.c` —— 驱动(里面有 `file_operations` 的 `open/read/write`)。
- `ledApp.c` —— 应用(`open /dev/led ,write` 开关灯)。

它们本来就是**分开的两个程序**。我们这次的 `read_raw` 相当于 `led.c` 里的 `.read` (都是驱动侧);区别只是——**我们没写配套 App,直接用 `cat` 顶替了那个应用**。

4.2 一次 `cat` 的完整数据流

```
用户敲: cat /sys/bus/iio/devices/iio:device0/in_accel_x_raw
```

↓ 系统调用 `read()` (用户态 → 内核态)

↓ IIO 子系统: 把请求转成对你 `read_raw()` 的调用

↓ 你的驱动 `read_raw():regmap` 读芯片的加速度寄存器

↓ 返回数值

```
用户屏幕上看到: 4660
```

看这条链: `cat` 属于应用, `read_raw` 属于驱动, 二者在用户态/内核态两边, 通过“读文件”这个标准动作连起来。它们从不“写在一起”。

五、用 MPU6050 的代码把三件事对上号

代码	对应哪件事
<code>id_table / of_match_table</code>	① 匹配:让内核把硬件交给你,触发 probe
<code>devm_regmap_init_i2c</code> (建通道)	② 初始化:建立读写芯片的通道
读 <code>WHO_AM_I</code> (0x75)	② 初始化:确认芯片身份
清 <code>PWR_MGMT_1</code> 的 sleep 位	② 初始化:唤醒芯片
<code>iio_chan_spec</code> 通道数组	③ 开接口:声明能测什么量(生成 <code>in_accel_x_raw</code>)
<code>read_raw()</code> 回调	③ 开接口:用户来读时去芯片取数据返回
<code>devm_iio_device_register</code>	③ 开接口:把这个"柜台"挂到 <code>/sys/bus/iio/</code> 下

六、为什么要这样分(驱动/应用解耦的好处)

- **解耦**:应用不用懂 I2C、不用懂芯片寄存器,只管读文件;硬件细节全被驱动挡在下面。
- **标准化**:换一颗别的加速度芯片,只要驱动也按 IIO 标准写,上层应用**一行不用改**。
- **通用**:任何语言、任何工具(cat / python / C / 图形程序)都能用同一种方式读它。

一句话收尾:**驱动不是"告诉内核有这个硬件",而是"让这个硬件真正能被用起来";而它的应用,是另一个分开的程序。**

本文为 MPU-6050 IIO 驱动学习的概念梳理。配套代码:github.com/fyy0619-cell/xiangmu